```
      image: my-image:latest
      ports:
containerPort: 80
```

This tells Kubernetes: ‘Run 3 copies of my app using this image, and expose port 80’.

Docker + Kubernetes = Cloud-native power

Kubernetes doesn’t replace Docker—it builds on top of it. You still use Docker (or an alternative like containerd) to create and run containers, but Kubernetes helps you manage them at scale, across clusters, in cloud or on-premises environments.

Whether you’re deploying to AWS, Azure, Google Cloud, or your own servers, Kubernetes helps ensure your app stays up, available, and adaptable.

Modern DevOps and GitOps with containers

Now that we’ve covered how containers and Kubernetes work together, let’s look at how they fit into the bigger picture of software delivery. In today’s fast-moving tech world, developers need ways to ship code quickly, safely, and repeatedly. That’s where DevOps and GitOps come in.

DevOps is more than just a buzzword—it’s a mindset and a set of practices that bring developers and operations teams together. The goal? Deliver software faster and with fewer bugs.

With containers, DevOps becomes even smoother because your app and its environment are bundled into a portable unit that behaves the same everywhere.

Containers in the CI/CD pipeline

CI/CD stands for continuous integration and continuous deployment. Here’s how containers fit into that flow:

- The developer writes code and pushes it to GitHub.
- A tool like GitHub Actions, Jenkins, or GitLab CI kicks off a pipeline.
- The app is built into a Docker image.
- The image is tested, scanned for vulnerabilities, and pushed to a Docker registry.
- Kubernetes pulls the latest image and updates the app automatically.

With this setup, you can go from code change to live update in production in minutes, with full automation and traceability.

If DevOps is the foundation, GitOps is the next level. Instead of manually configuring your servers or clusters, GitOps treats your infrastructure as code. Here’s how it works:

- All your app configuration and Kubernetes deployment files are stored in Git.
- A tool like ArgoCD or Flux watches your Git repo.
- When changes are committed (e.g., a new version of your app), the tool automatically syncs those changes to your Kubernetes cluster.

In other words, Git becomes the source of truth, and your cluster always reflects what’s in your Git repo.

A simple example workflow

Let’s say you’re working on a web app. Here’s what your modern DevOps + GitOps workflow may look like:

- You push an update to GitHub.
- GitHub Actions builds and tests your Docker image, then pushes it to Docker Hub.
- Your deployment YAML file in Git is updated with the new image tag.
- ArgoCD sees the change and updates your Kubernetes deployment.
- A few seconds later, your app is live with the new changes—no manual steps needed!

This kind of automation is incredibly powerful, especially for teams working with microservices or multiple environments (like dev, staging, and production).

With containers, Kubernetes, and GitOps practices, teams can ship features faster, recover from failures quickly, track every change, and scale with confidence. It’s a big part of why container-native DevOps is becoming the new normal across industries.

Over the past few years, the way we build and run software has changed dramatically—and containers have been at the heart of that shift. What started with virtual machines has now evolved into something faster, lighter, and far more adaptable. Containers, with tools like Docker, have made it easier than ever to develop, test, and ship applications across different environments. And with Kubernetes, managing those containers at scale has become not only possible but manageable.

Whether you’re a developer just starting out, a student exploring infrastructure, or a seasoned engineer looking to modernise workflows, containers offer an exciting path forward. Start small—try Docker on your laptop, deploy a test app, and write your first Kubernetes YAML file. The tools are open, the communities are helpful, and the opportunities are endless.

So go ahead—pull an image, spin up a container, and take your first step into the world of modern infrastructure. END

By: Meghraj Singh Beniwal

The author is the founder of Schooling for Next Generation, a tech enthusiast and Android hobbyist. He is a test automation analyst working in Virginia in the US.